

Arrays, Linked Lists and Recursion

Algorithms and Data Structures

Arrays

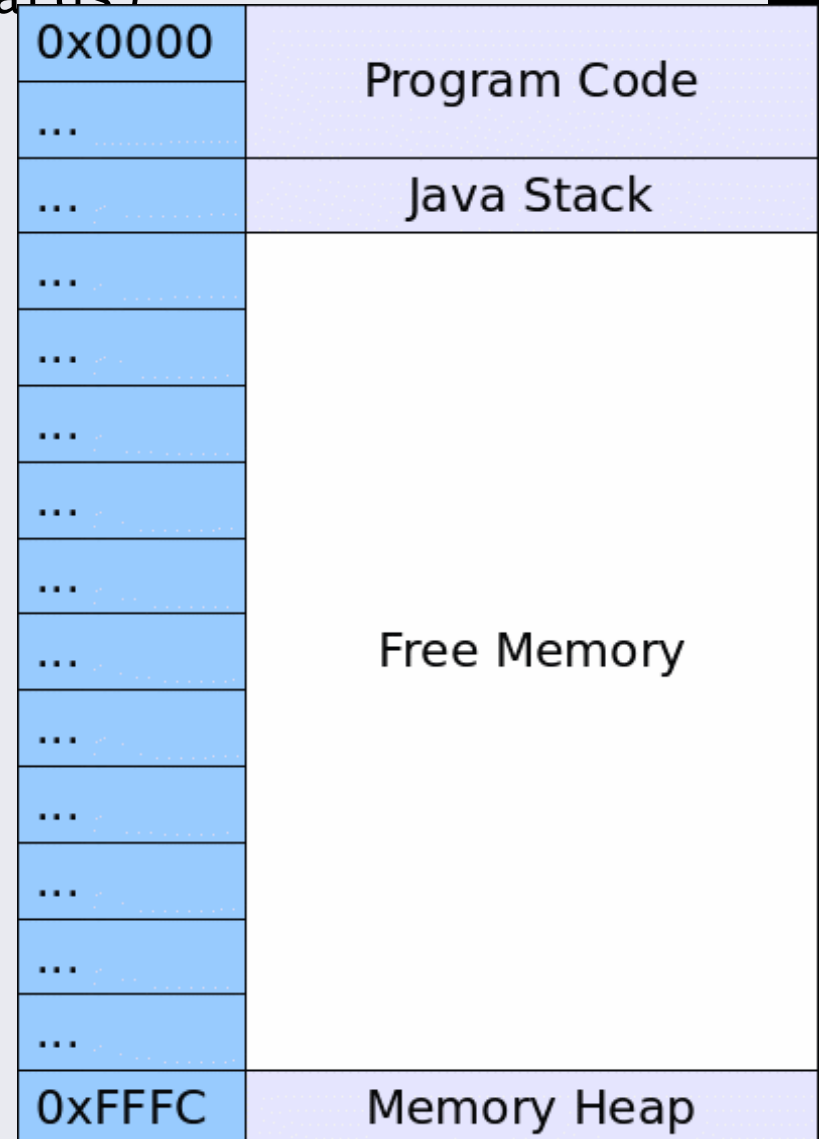
- Data structure consisting of a group of elements having a single name that are accessed by **indexing**.
 - computer science definition of an **array**
- Occupies a **contiguous** area of storage.
 - most programming languages
- Each element has the **same data type**.
 - **statically typed** programming languages

Java Example 1

```
1 public class Example1
2 {
3     public static void main(String[] args)
4     {
5         int[] array = new int[4];
6         array[0] = 255;
7     }
8 }
```

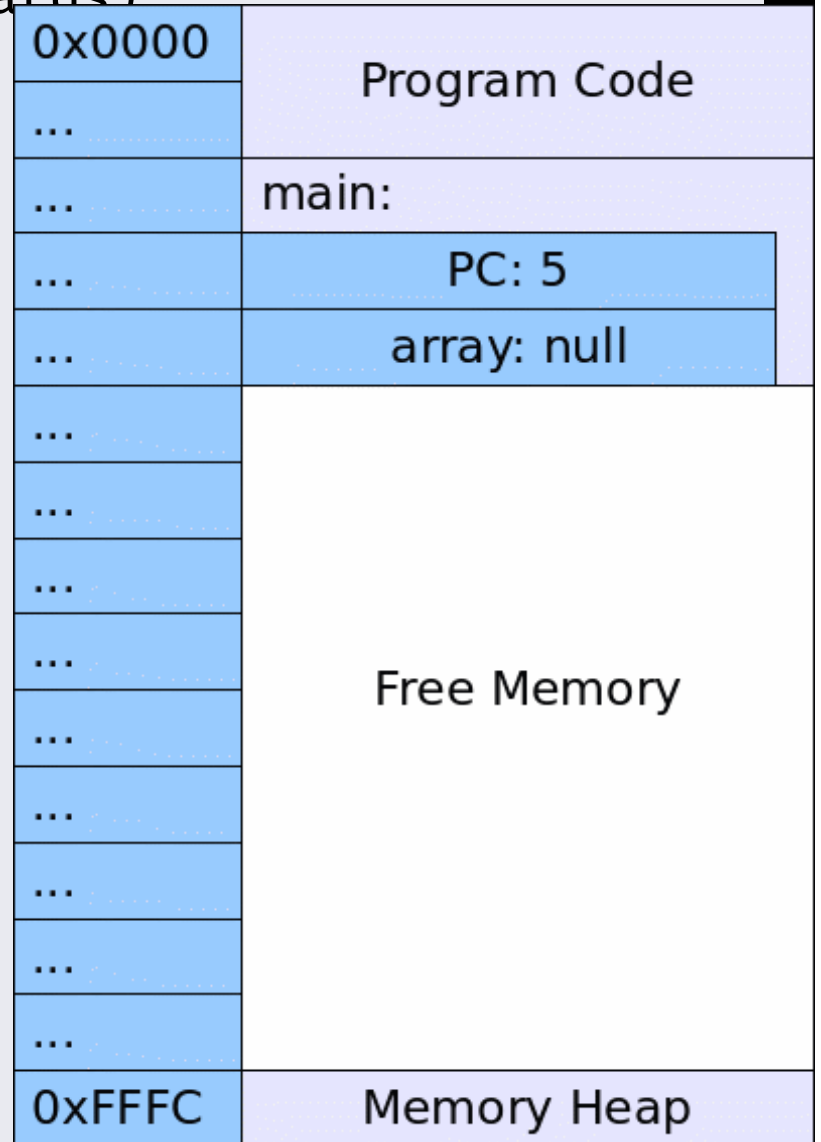
Java Example 1

```
1 public class Example1
2 {
3     public static void main(String[] args)
4     {
5         int[] array = new int[4];
6         array[0] = 255;
7     }
8 }
```



Java Example 1

```
1 public class Example1
2 {
3     public static void main(String[] args)
4     {
5         int[] array = new int[4];
6         array[0] = 255;
7     }
8 }
```



Java Example 1

```
1 public class Example1
2 {
3     public static void main(String[] args)
4     {
5         int[] array = new int[4];
6         array[0] = 255;
7     }
8 }
```

0x0000	Program Code
...	
...	main:
...	
...	
...	PC: 6
...	array: 0xFFEC
...	Free Memory
...	
...	
...	
...	
0xFFEC	4
0xFFFF0	0
0xFFFF4	0
0xFFFF8	0
0xFFFFC	0

Array size

Array storage

Java Example 1

```
1 public class Example1
2 {
3     public static void main(String[] args)
4     {
5         int[] array = new int[4];
6         array[0] = 255;
7     }
8 }
```

0x0000	Program Code
...	
...	main:
...	
...	
...	PC: 7
...	array: 0xFFEC
...	
...	Free Memory
...	
...	
...	
...	
0xFFEC	4
0xFFFF0	255
0xFFFF4	0
0xFFFF8	0
0xFFFFC	0

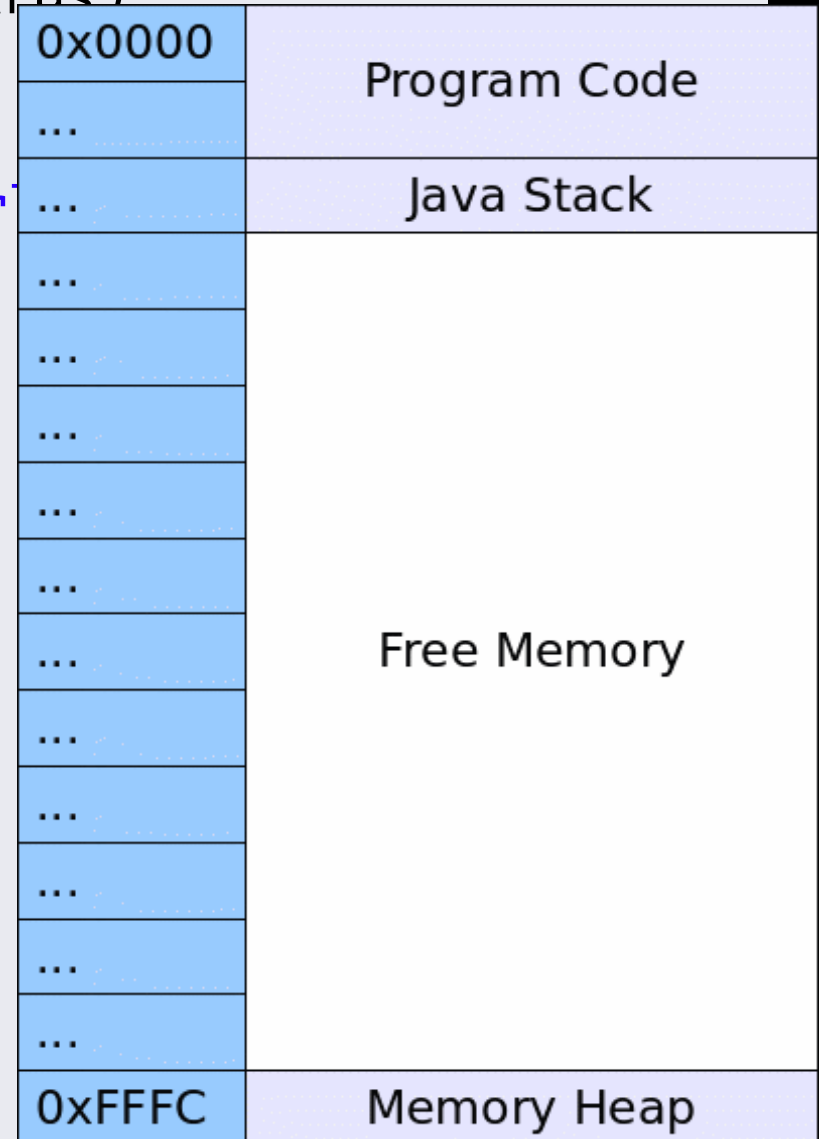
array[0]

Java Example 2

```
1 public class Example2
2 {
3     public static void main(String[] args)
4     {
5         String[] array = new String[4];
6         array[0] = new String("Hello world");
7         System.out.println(array[0]);
8     }
9 }
```

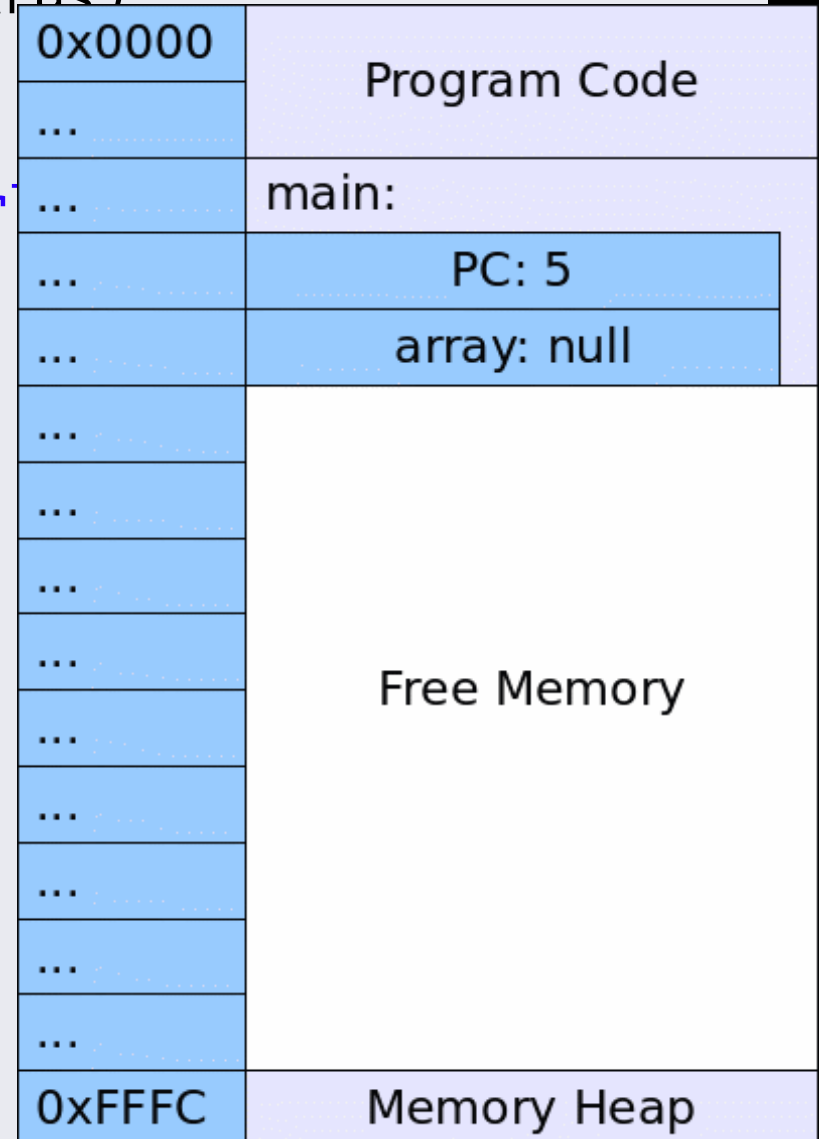

Java Example 2

```
1 public class Example2
2 {
3     public static void main(String[] args)
4     {
5         String[] array = new String[4];
6         array[0] = new String("Hello world");
7         System.out.println(array[0]);
8     }
9 }
```



Java Example 2

```
1 public class Example2
2 {
3     public static void main(String[] args)
4     {
5         String[] array = new String[4];
6         array[0] = new String("Hello world");
7         System.out.println(array[0]);
8     }
9 }
```



Java Example 2

```
1 public class Example2
2 {
3     public static void main(String[] args)
4     {
5         String[] array = new String[4];
6         array[0] = new String("Hello world");
7         System.out.println(array[0]);
8     }
9 }
```

0x0000	Program Code
...	
...	main:
...	
...	
...	PC: 6
...	array: 0xFFEC
...	Free Memory
...	
...	
...	
...	
0xFFEC	4
0xFFFF0	null
0xFFFF4	null
0xFFFF8	null
0xFFFFC	null

Java Example 2

```
1 public class Example2
2 {
3     public static void main(String[] args)
4     {
5         String[] array = new String[4];
6         array[0] = new String("Hello wor
7         System.out.println(array[0]);
8     }
9 }
```

0xFFD0	11	
0xFFD4	'H'	'e'
0xFFD8	'l'	'l'
0xFFDC	'o'	' '
0xFFE0	'w'	'o'
0xFFE4	'r'	'l'
0xFFE8	'd'	'\0'

Dynamically allocated

0x0000	Program Code	
...		
...	main:	
...	PC: 6	
	array: 0xFFEC	
	Free Memory	
	11	
	'H'	'e'
	...	...
	'd'	'\0'
FFEC	4	
0xFFFF0	null	
0xFFFF4	null	
0xFFFF8	null	
0xFFFFC	null	

Java Example 2

```
1 public class Example2
2 {
3     public static void main(String[] args)
4     {
5         String[] array = new String[4];
6         array[0] = new String("Hello wor
7         System.out.println(array[0]);
8     }
9 }
```

0x0000	Program Code	
...		
...	main:	
...	PC: 7	
...	array: 0xFFEC	
...	Free Memory	
0xFFD0	11	
...	'H'	'e'
...	...	...
...	'd'	'\0'
0xFFEC	4	
0xFFFF0	0xFFD0	
0xFFFF4	null	
0xFFFF8	null	
0xFFFC	null	

Reference

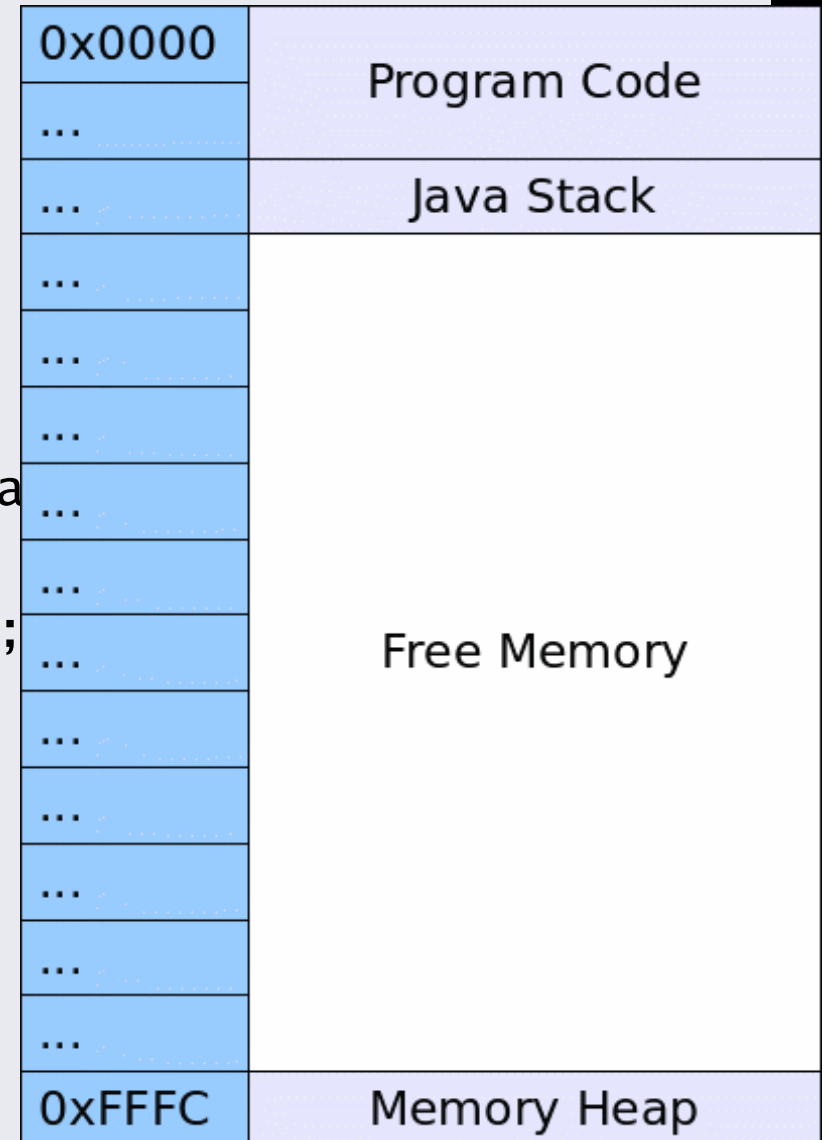


Java Example 3

```
1  public class Array
2  {
3      private Object[] data;
4
5      public Array()
6      {
7          this.data = new Object[4];
8      }
9
10     public static void main(String[] args)
11     {
12         Array arrayObject = new Array();
13     }
14 }
```

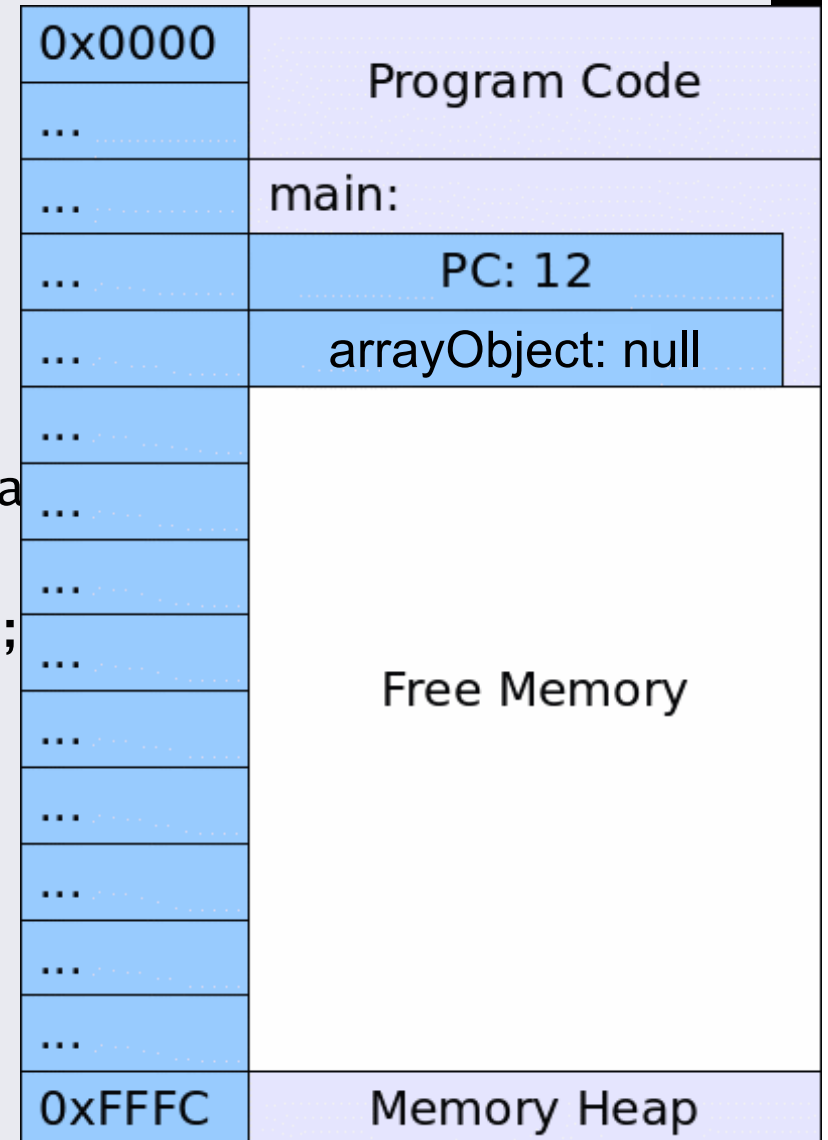
Java Example 3

```
1 public class Array
2 {
3     private Object[] data;
4
5     public Array()
6     {
7         this.data = new Object[4];
8     }
9
10    public static void main(String[] a
11    {
12        Array arrayObject = new Array();
13    }
14 }
```



Java Example 3

```
1 public class Array
2 {
3     private Object[] data;
4
5     public Array()
6     {
7         this.data = new Object[4];
8     }
9
10    public static void main(String[] a
11    {
12        Array arrayObject = new Array();
13    }
14 }
```



Java Example 3

```
1  public class Array
2  {
3      private Object[] data;
4
5      public Array()
6      { //default constructor
7          this.data = new Object[4];
8      }
9
10     public static void main(String[] a
11     {
12         Array arrayObject = new Array();
13     }
14 }
```

0x0000	Program Code
...	
...	main:
...	PC: 12
...	arrayObject: null
...	Array:
...	PC: 7
...	this: 0xFFFC
...	Free Memory
...	
...	
...	
...	
0xFFFC	data: null

Java Example 3

```
1 public class Array
2 {
3     private Object[] data;
4
5     public Array()
6     {
7         this.data = new Object[4];
8     }
9
10    public static void main(String[] a
11    {
12        Array arrayObject = new Array();
13    }
14 }
```

0x0000	Program Code
...	
...	main:
...	PC: 12
...	arrayObject: null
...	Array:
...	PC: 8
...	this: 0xFFFC
...	Free Memory
0xFFE8	4
0xFFEC	null
0xFFFF0	null
0xFFFF4	null
0xFFFF8	null
0xFFFC	data: 0xFFE8

Java Example 3

```
1 public class Array
2 {
3     private Object[] data;
4
5     public Array()
6     {
7         this.data = new Object[4];
8     }
9
10    public static void main(String[] a
11    {
12        Array arrayObject = new Array();
13    }
14 }
```

0x0000	Program Code
...	
...	main:
...	PC: 12
...	arrayObject: null
...	0xFFFC
...	Free Memory
...	
...	
...	
0xFFE8	4
0xFFEC	null
0xFFFF0	null
0xFFFF4	null
0xFFFF8	null
0xFFFC	data: 0xFFE8

Java Example 3

```
1 public class Array
2 {
3     private Object[] data;
4
5     public Array()
6     {
7         this.data = new Object[4];
8     }
9
10    public static void main(String[] a
11    {
12        Array arrayObject = new Array();
13    }
14 }
```

0x0000	Program Code
...	
...	main:
...	PC: 13
...	arrayObject: 0xFFFC
...	Free Memory
...	
...	
...	
0xFFE8	4
0xFFEC	null
0xFFFF0	null
0xFFFF4	null
0xFFFF8	null
0xFFFC	data: 0xFFE8

Contiguous Uniform Storage

- What does it get us?
- What does it cost us?

Arrays (insert)

// sorted capital cities

0	0x...	→	'Brisbane'
1	0x...	→	'Darwin'
2	0x...	→	'Hobart'
3	0x...	→	'Melbourne'
4	0x...	→	'Perth'
5	0x...	→	'Sydney'
6	null		
7	null		
8	null		
9	null		

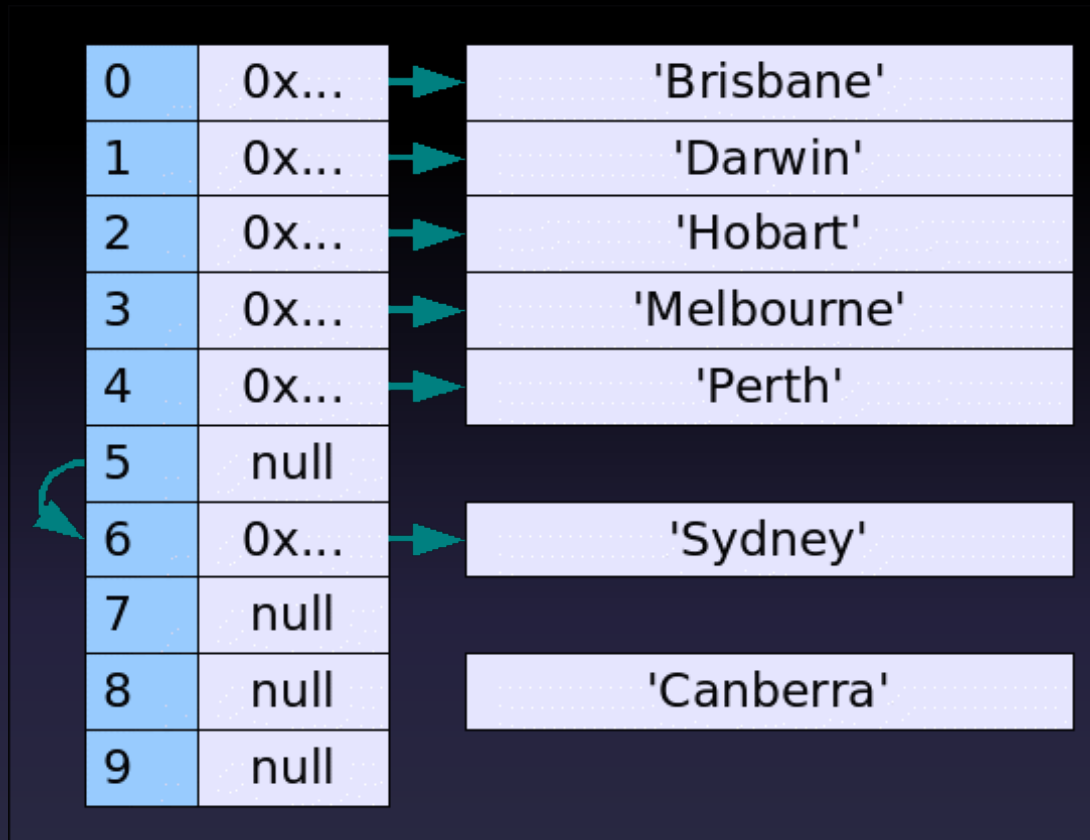
Arrays (insert)

0	0x...	→	'Brisbane'
1	0x...	→	'Darwin'
2	0x...	→	'Hobart'
3	0x...	→	'Melbourne'
4	0x...	→	'Perth'
5	0x...	→	'Sydney'
6	null		
7	null		
8	null		
9	null		

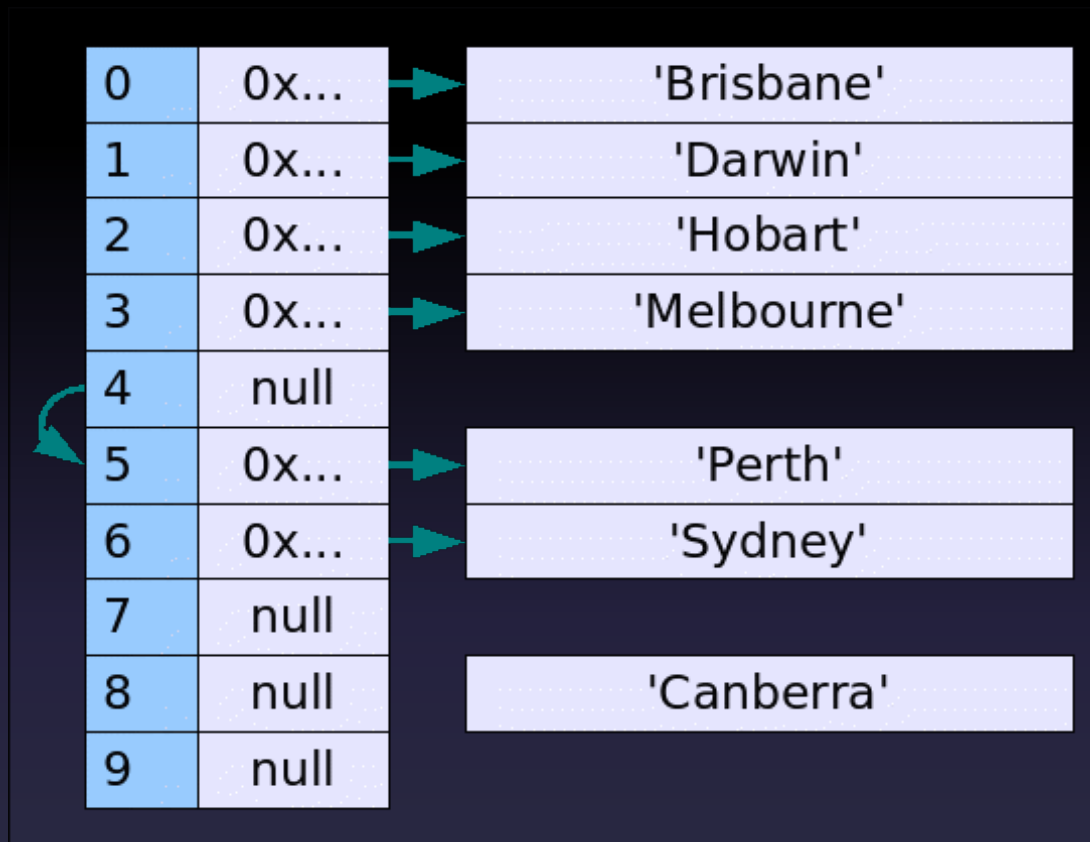
// Oops, we forgot Canberra,
// Let's fix it

'Canberra'

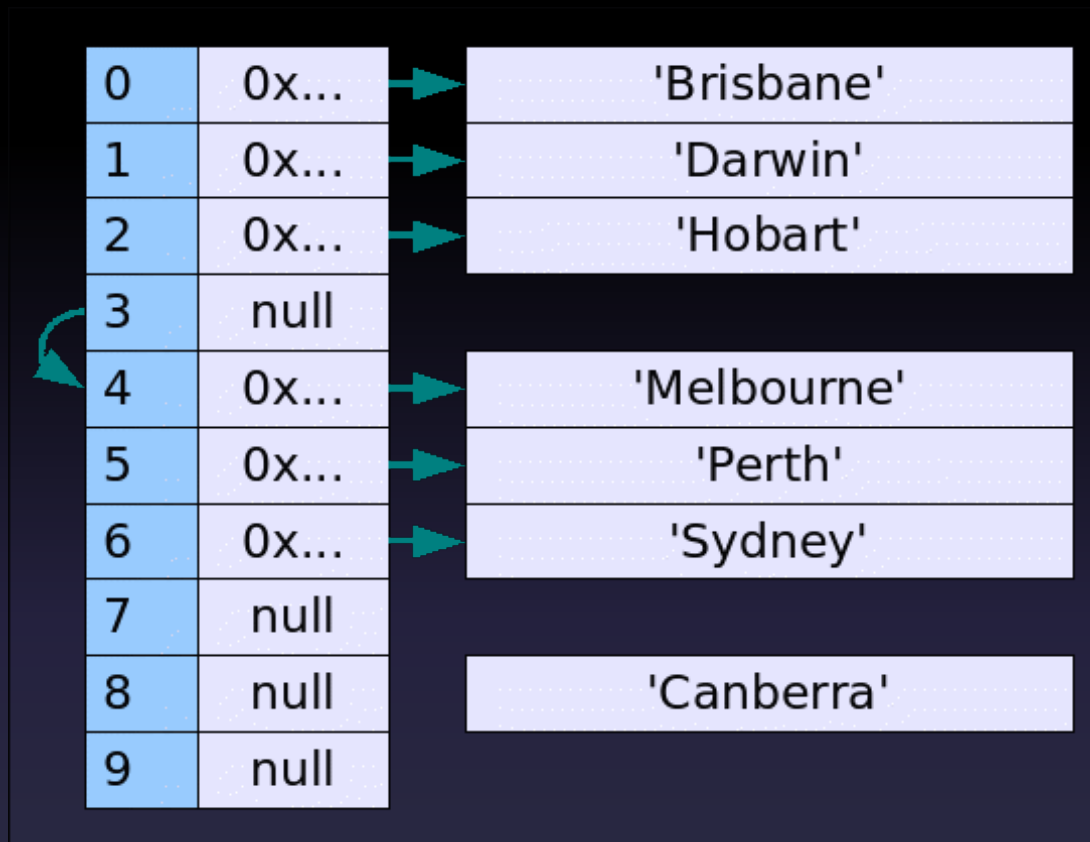
Arrays (insert)



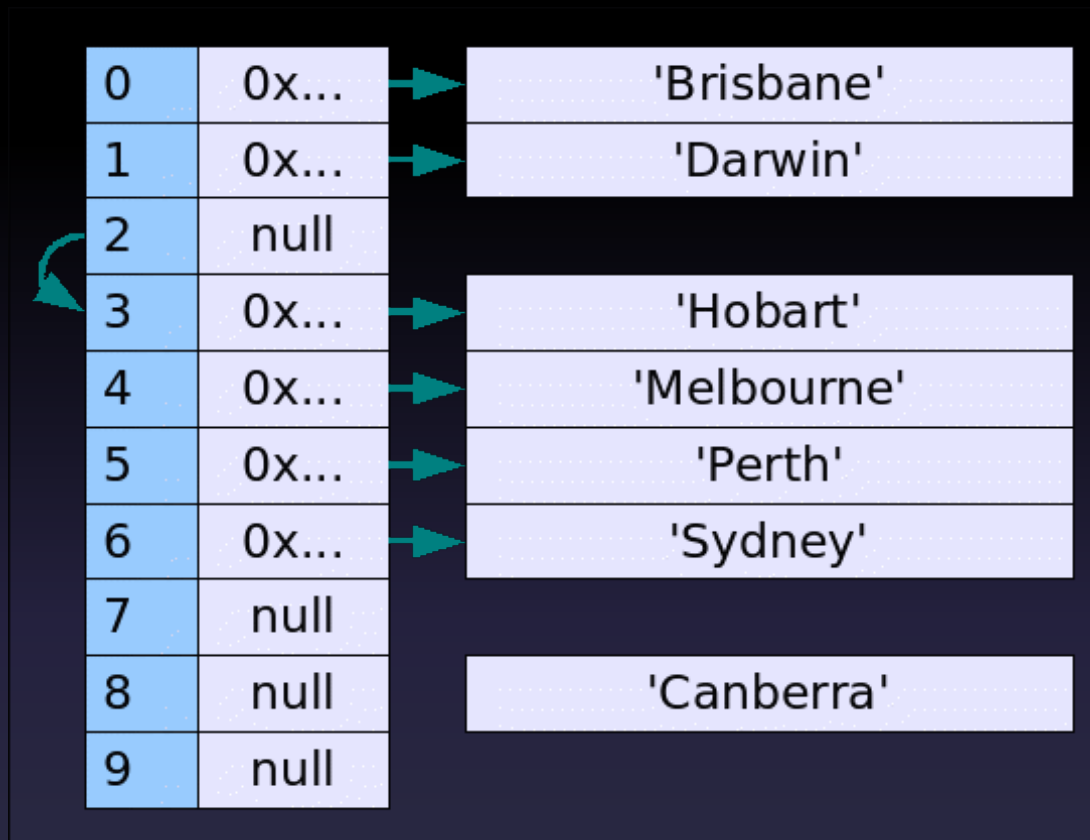
Arrays (insert)



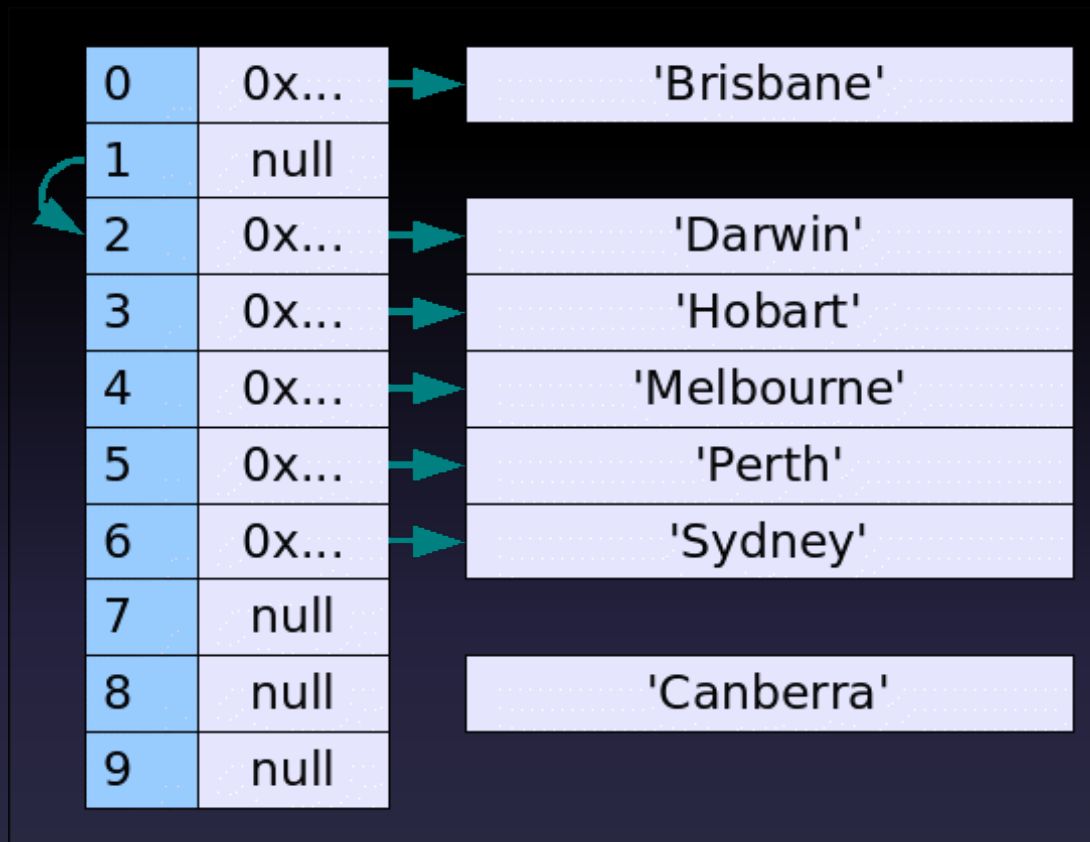
Arrays (insert)



Arrays (insert)



Arrays (insert)



Arrays (insert)

// sorted capital cities

0	0x...	→	'Brisbane'
1	0x...	→	'Canberra'
2	0x...	→	'Darwin'
3	0x...	→	'Hobart'
4	0x...	→	'Melbourne'
5	0x...	→	'Perth'
6	0x...	→	'Sydney'
7	null		
8	null		
9	null		

Arrays Summary

- Pros
 - Constant time access to indexed memory location
- Cons
 - Fixed size
 - Resizing requires copying all existing values to new array
 - Sorted insert
 - Best case constant
 - Worst case, must shuffle entire array of n elements.

Linked Lists

- Dynamic
 - Heap provides a dynamic supply of memory for new objects.
- Incremental
 - Chained data structure is used to allow incremental growth.

LinkedList and ListNode

```
class ListNode<T> {  
    T element;  
    ListNode<T> next;  
}
```

```
public class LinkedList<T> {  
    private ListNode<T> head; // location of head  
    private ListNode<T> tail; // location of tail  
    private int size;  
    ...  
}
```

Why do we have 'size'?



Note: This creates reference variables called head and tail.
Both are initially NULL; no node has been created.

LinkedList (addFirst)

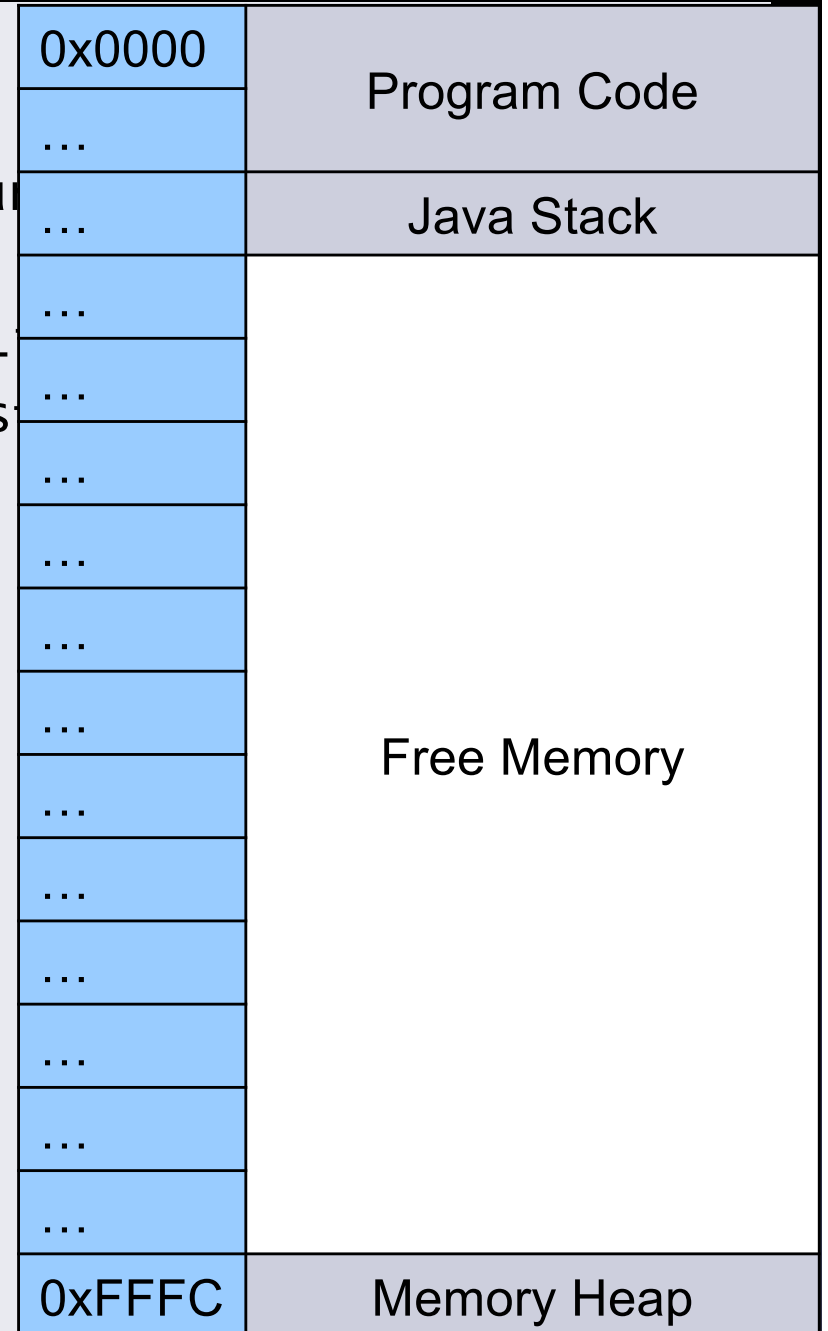
```
public class LinkedList<T> {  
    private ListNode<T> head;  
    private ListNode<T> tail;  
    private int size;  
  
    public void addFirst(ListNode<T> aNode) {  
        aNode.next = this.head; // link new node to previous head  
        this.head = aNode; // aNode becomes head of the list  
  
        if (this.tail == null) {  
            // if list is empty then make aNode the tail as well  
            this.tail = aNode;  
        }  
        this.size++;  
    }  
}  
  
// note: addFirst means make aNode be first in the list
```

Java Example 4

```
1  // concrete example using strings
2
3  public class Example4
4  {
5      public static void main(String[] args)
6      {
7          // declare new list and new node
8          LinkedList<Integer> list = new LinkedList<>();
9          ListNode<Integer> node = new ListNode<>();
10
11         // assign value to node element
12         node.element = 11;
13         // add node to the list
14         list.addFirst(node);
15     }
16 }
```

Java Example 4

```
1 public class Example4
2 {
3     public static void main(String[] args)
4     {
5         LinkedList<Integer> list = new LinkedList<>();
6         ListNode<Integer> node = new ListNode<>(11);
7
8         node.element = 11;
9
10        list.addFirst(node);
11    }
12 }
```



Java Example 4

```
1 public class Example4
2 {
3     public static void main(String[] args)
4     {
5         LinkedList<Integer> list = new LinkedList<>();
6         ListNode<Integer> node = new ListNode<>(11);
7
8         node.element = 11;
9
10        list.addFirst(node);
11    }
12 }
```

[illegible]

Java Example 4

```
1 public class Example4
2 {
3     public static void main(String[] args)
4     {
5         LinkedList<Integer> list = new LinkedList<>();
6         ListNode<Integer> node = new ListNode<>(11);
7
8         node.element = 11;
9
10        list.addFirst(node);
11    }
12 }
```

list: LinkedList
head
tail
size: 0

0x0000	Program Code
...	
...	main():
...	PC: 6
...	list: 0xFFF4
...	node: null
...	Free Memory
...	
...	
...	
...	
...	
0xFFF4	head: null
0xFFF8	tail: null
0xFFFC	size: 0

Java Example 4

```
1 public class Example4
2 {
3     public static void main(String[] args)
4     {
5         LinkedList<Integer> list = new LinkedList<>();
6         ListNode<Integer> node = new ListNode<>();
7
8         node.element = 11;
9
10        list.addFirst(node);
11    }
12 }
```

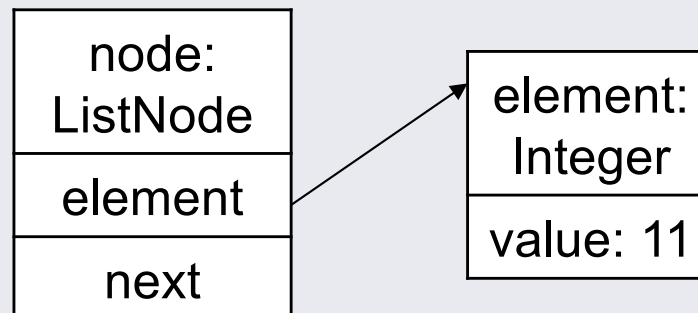
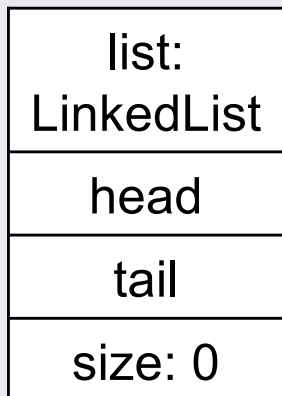
list: LinkedList
head
tail
size: 0

node: ListNode
element
next

0x0000	Program Code
...	
...	main(): PC: 8 list: 0xFFFF4 node: 0xFFEC
...	
...	
...	
...	Free Memory
...	
...	
...	
0xFFEC	element: null
0xFFFF0	next: null
0xFFFF4	head: null
0xFFFF8	tail: null
0xFFFFC	size: 0

Java Example 4

```
1 public class Example4
2 {
3     public static void main(String[] args)
4     {
5         LinkedList<Integer> list = new LinkedList<>();
6         ListNode<Integer> node = new ListNode<>();
7
8         node.element = 11;
9
10        list.addFirst(node);
11    }
12 }
```

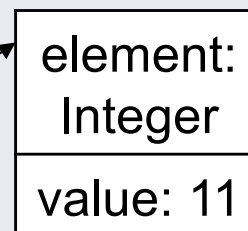
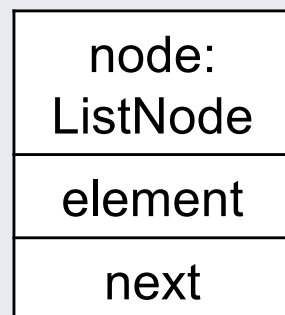
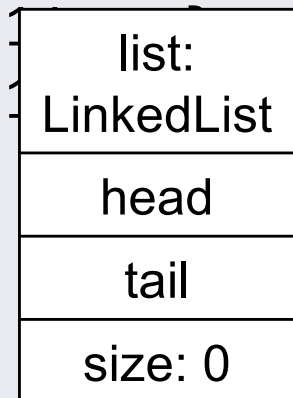


0x0000	Program Code
...	
...	main():
...	PC: 10
...	list: 0xFFF4
...	node: 0xFFEC
...	Free Memory
...	
...	
...	
0xFFE8	11
0xFFEC	element: 0xFFE8
0xFFFF0	next: null
0xFFF4	head: null
0xFFFF8	tail: null
0xFFFFC	size: 0

Java Example 4

```

1  public class LinkedList<T> {
2      private ListNode<T> head;
3      private ListNode<T> tail;
4      private int size;
5
6      public void addFirst(ListNode<T> aNode) {
7          aNode.next = this.head;
8          this.head = aNode;
9
10         if (this.tail == null) {
11             this.tail = aNode;
12         }
13         this.size++;
    
```



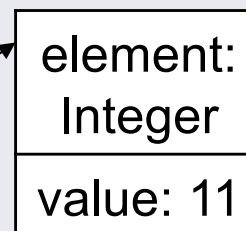
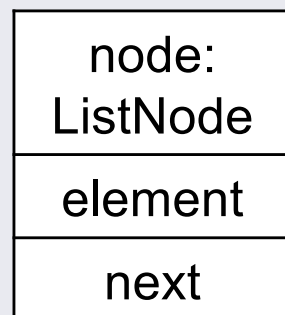
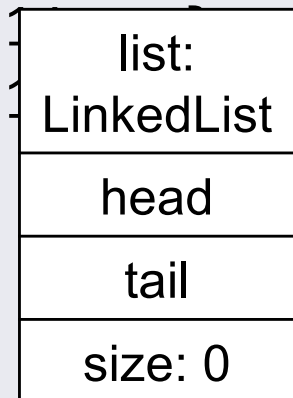
this aNode

0x0000	Program Code
...	
...	
...	main():
...	PC: 10
...	list: 0xFFF4
...	node: 0xFFEC
...	addFirst():
...	PC: 7
...	this: 0xFFF4
...	aNode: 0xFFEC
	Free Memory
0xFFE8	11
0xFFEC	element: 0xFFE8
0xFFFF0	next: null
0xFFF4	head: null
0xFFFF8	tail: null
0xFFFFC	size: 0

Java Example 4

```
1 public class LinkedList<T> {
2     private ListNode<T> head;
3     private ListNode<T> tail;
4     private int size;
5
6     public void addFirst(ListNode<T> aNode) {
7         aNode.next = this.head;
8         this.head = aNode;
9
10        if (this.tail == null) {
11            this.tail = aNode;
12        }
13        this.size++;

```



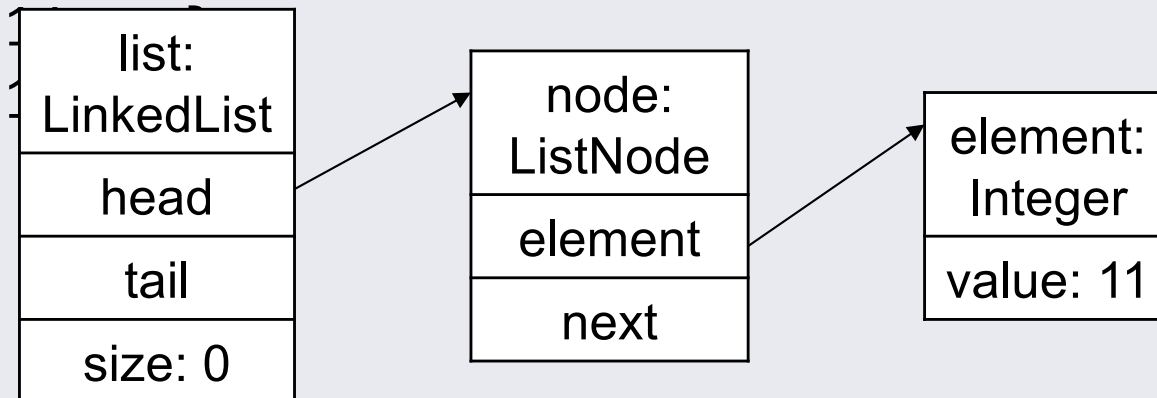
0x0000	Program Code
...	
...	
...	main():
...	PC: 10
...	list: 0xFFF4
...	node: 0xFFEC
...	addFirst():
...	PC: 8
...	this: 0xFFF4
...	aNode: 0xFFEC
	Free Memory
0xFFE8	11
0xFFEC	element: 0xFFE8
0xFFFF0	next: null
0xFFF4	head: null
0xFFFF8	tail: null
0xFFFFC	size: 0

Java Example 4

```

1  public class LinkedList<T> {
2      private ListNode<T> head;
3      private ListNode<T> tail;
4      private int size;
5
6      public void addFirst(ListNode<T> aNode) {
7          aNode.next = this.head;
8          this.head = aNode;
9
10         if (this.tail == null) {
11             this.tail = aNode;
12         }
13         this.size++;

```



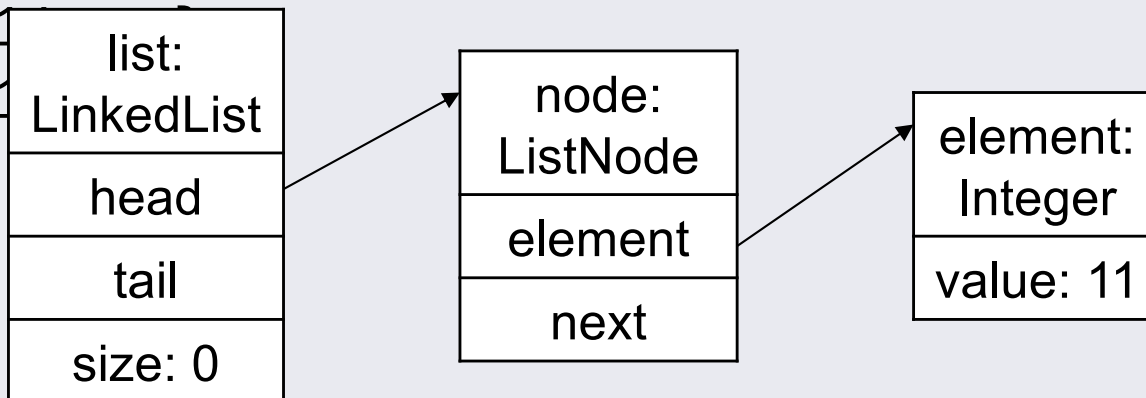
0x0000	Program Code
...	
...	
...	main():
...	PC: 10
...	list: 0xFFF4
...	node: 0xFFEC
...	addFirst():
...	PC: 10
...	this: 0xFFF4
...	aNode: 0xFFEC
	Free Memory
0xFFE8	11
0xFFEC	element: 0xFFE8
0xFFF0	next: null
0xFFF4	head: 0xFFEC
0xFFF8	tail: null
0xFFFC	size: 0

Java Example 4

```

1  public class LinkedList<T> {
2      private ListNode<T> head;
3      private ListNode<T> tail;
4      private int size;
5
6      public void addFirst(ListNode<T> aNode) {
7          aNode.next = this.head;
8          this.head = aNode;
9
10         if (this.tail == null) {
11             this.tail = aNode;
12         }
13         this.size++;

```



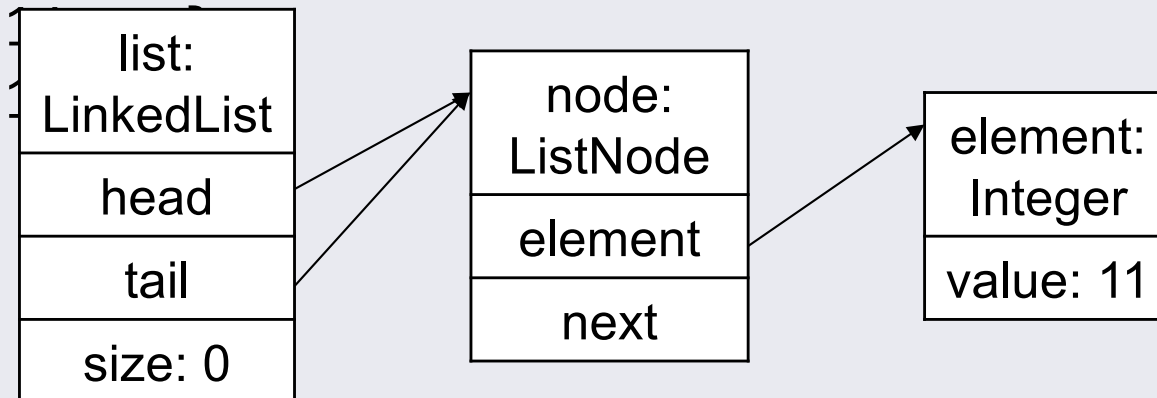
0x0000	Program Code
...	
...	
...	main():
...	PC: 10
...	list: 0xFFF4
...	node: 0xFFEC
...	addFirst():
...	PC: 11
...	this: 0xFFF4
...	aNode: 0xFFEC
	Free Memory
0xFFE8	11
0xFFEC	element: 0xFFE8
0xFFFF0	next: null
0xFFF4	head: 0xFFEC
0xFFFF8	tail: null
0xFFFFC	size: 0

Java Example 4

```

1  public class LinkedList<T> {
2      private ListNode<T> head;
3      private ListNode<T> tail;
4      private int size;
5
6      public void addFirst(ListNode<T> aNode) {
7          aNode.next = this.head;
8          this.head = aNode;
9
10         if (this.tail == null) {
11             this.tail = aNode;
12         }
13         this.size++;

```



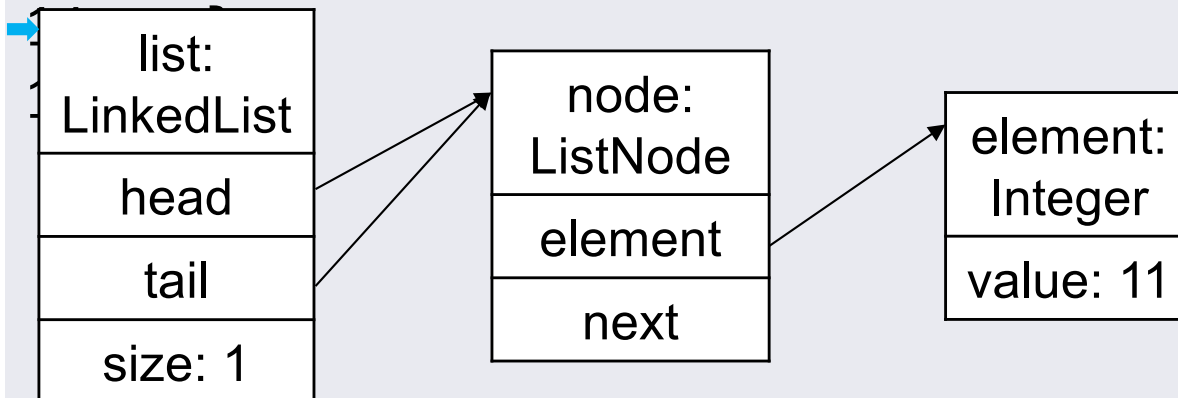
0x0000	Program Code
...	
...	
...	main():
...	PC: 10
...	list: 0xFFF4
...	node: 0xFFEC
...	addFirst():
...	PC: 13
...	this: 0xFFF4
...	aNode: 0xFFEC
	Free Memory
0xFFE8	11
0xFFEC	element: 0xFFE8
0xFFFF0	next: null
0xFFF4	head: 0xFFEC
0xFFFF8	tail: 0xFFEC
0xFFFFC	size: 0

Java Example 4

```

1  public class LinkedList<T> {
2      private ListNode<T> head;
3      private ListNode<T> tail;
4      private int size;
5
6      public void addFirst(ListNode<T> aNode) {
7          aNode.next = this.head;
8          this.head = aNode;
9
10         if (this.tail == null) {
11             this.tail = aNode;
12         }
13         this.size++;

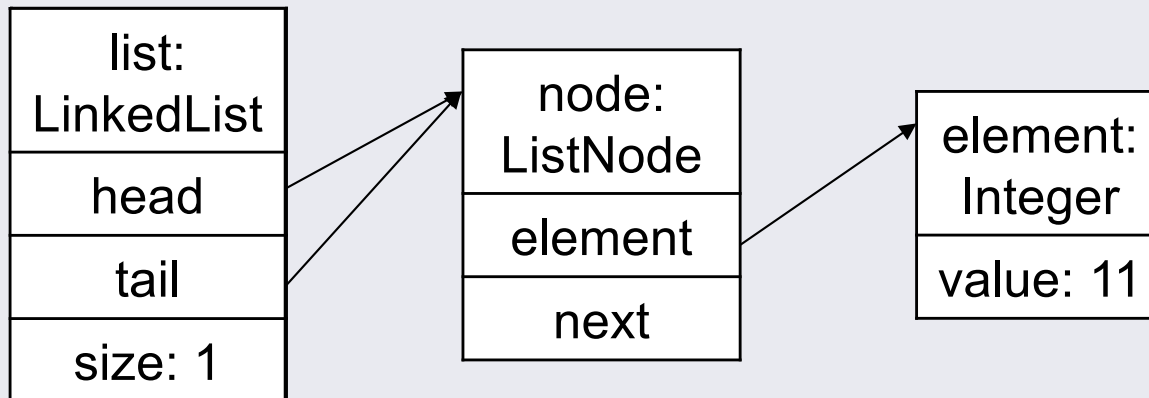
```



0x0000	Program Code
...	
...	main():
...	PC: 10
...	list: 0xFFF4
...	node: 0xFFEC
...	addFirst():
...	PC: 14
...	this: 0xFFF4
...	aNode: 0xFFEC
	Free Memory
0xFFE8	11
0xFFEC	element: 0xFFE8
0xFFFF0	next: null
0xFFF4	head: 0xFFEC
0xFFFF8	tail: 0xFFEC
0xFFFFC	size: 1

Java Example 4

```
1 public class Example4
2 {
3     public static void main(String[] args)
4     {
5         LinkedList<Integer> list = new LinkedList<>();
6         ListNode<Integer> node = new ListNode<>();
7
8         node.element = 11;
9
10        list.addFirst(node);
11    }
12 }
```



0x0000	Program Code
...	
...	main():
...	PC: 10
...	list: 0xFFF4
...	node: 0xFFEC
...	Free Memory
...	
...	
...	
0xFFE8	11
0xFFEC	element: 0xFFE8
0xFFFF0	next: null
0xFFF4	head: 0xFFEC
0xFFFF8	tail: 0xFFEC
0xFFFFC	size: 1

List: LinkedList

Head:

Tail:

Size: 4

Node:
ListNode

Element: 4

Next

Node:
ListNode

Element: 17

Next

Node:
ListNode

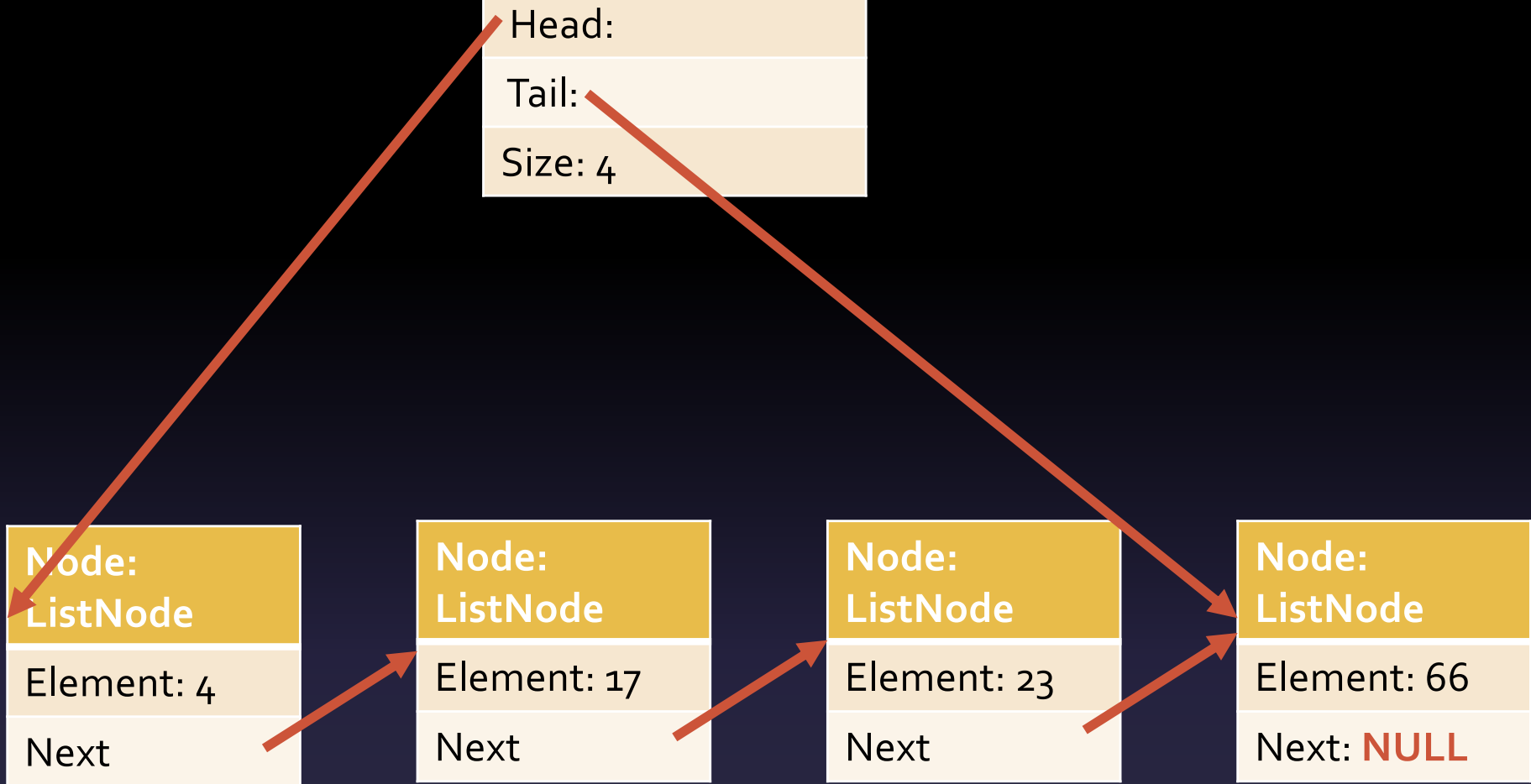
Element: 23

Next

Node:
ListNode

Element: 66

Next: **NULL**



Linked Lists Summary

- Do not support access via index
 - Must traverse entire list
- Singly linked lists
 - Can easily add to head and tail of list
 - Can easily remove from head of list
- Doubly linked lists **(what is it?)**
 - Can add to/remove from head and tail
 - Can insert anywhere you have a pointer to

Recursion

- Linear Recursion
- Binary Recursion
- Multiple Recursion

Recursion Pattern

- Recursion: when a method calls itself
- Classic example – factorial function:
 - $n! = 1 \times 2 \times 3 \times 4 \times \cdots \times (n - 1) \times n$
- Recursive definition: $f(n) = \begin{cases} 1, & \text{if } n = 0 \\ n \cdot f(n - 1), & \text{else} \end{cases}$

1.3 A Brief Introduction to Recursion

Most mathematical functions that we are familiar with are described by a simple formula. For instance, we can convert temperatures from Fahrenheit to Celsius by applying the formula

$$C = 5(F - 32)/9$$

Given this formula, it is trivial to write a Java method; with declarations and braces removed, the one-line formula translates to one line of Java.

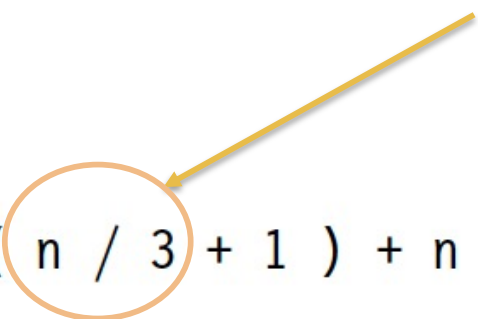
Mathematical functions are sometimes defined in a less standard form. As an example, we can define a function f , valid on nonnegative integers, that satisfies $f(0) = 0$ and $f(x) = 2f(x - 1) + x^2$. From this definition we see that $f(1) = 1$, $f(2) = 6$, $f(3) = 21$, and $f(4) = 58$. A function that is defined in terms of itself is called **recursive**. Java allows functions to be recursive.¹ It is important to remember that what Java provides is merely an attempt to follow the recursive spirit. Not all mathematically recursive functions are efficiently (or correctly) implemented by Java's simulation of recursion. The idea is that the recursive function f ought to be expressible in only a few lines, just like a nonrecursive function. Figure 1.2 shows the recursive implementation of f .

$$f(0) = 0 \quad f(x) = 2f(x - 1) + x^2.$$

```
1      public static int f( int x )
2      {
3          if( x == 0 )                // Base case
4              return 0;
5          else
6              return 2 * f( x - 1 ) + x * x;
7      }
```

Figure 1.2 A recursive method

```
1    public static int bad( int n )
2    {
3        if( n == 0 )
4            return 0;
5        else
6            return bad( n / 3 + 1 ) + n - 1;
7    }
```

A yellow arrow points from the text "Integer division" to the expression "n / 3" in the recursive call on line 6. The expression "n / 3" is circled in orange.

Integer division

Figure 1.3 A nonterminating recursive method

Printing integer (binary representation) as text

Suppose we have a positive integer, n , that we wish to print out. Our routine will have the heading `printOut(n)`. Assume that the only I/O routines available will take a single-digit number and output it to the terminal. We will call this routine `printDigit`; for example, `printDigit(4)` will output a 4 to the terminal.

Printing integer (binary representation) as text

Suppose we have a positive integer, n , that we wish to print out. Our routine will have the heading `printOut(n)`. Assume that the only I/O routines available will take a single-digit number and output it to the terminal. We will call this routine `printDigit`; for example, `printDigit(4)` will output a 4 to the terminal.

```
1      public static void printOut( int n )  /* Print nonnegative n */
2      {
3          if( n >= 10 )
4              printOut( n / 10 );
5          printDigit( n % 10 );
6      }
```

Figure 1.4 Recursive routine to print an integer

Linear Recursion

- Test for **base cases**
 - Begin by testing for a set of base cases
 - there should be at least one
 - Every possible chain of recursive calls *must* eventually reach a **base case**
 - Handling of each base case **should not use recursion**

Linear Recursion

- Perform a single recursive call
- Define each possible recursive call so that it makes progress towards a base case

```
public void recursiveMethod(int n) {  
    if (n <= 0) {  
        return 0;  
    } else if (n%2 == 0) {  
        return 1 + recursiveMethod(n/2);  
    } else {  
        return 1 + recursiveMethod(n-1);  
    }  
}
```

(Can you spot any bug?)

Linear Recursion

- Perform a single recursive call
- Define each possible recursive call so that it makes progress towards a base case

```
public void recursiveMethod(int n) {  
    if (n <= 0) {  
        return 0;  
    } else if (n%2 == 0) {  
        return 1 + recursiveMethod(n/2);  
    } else {  
        return 1 + recursiveMethod(n-1);  
    }  
}
```

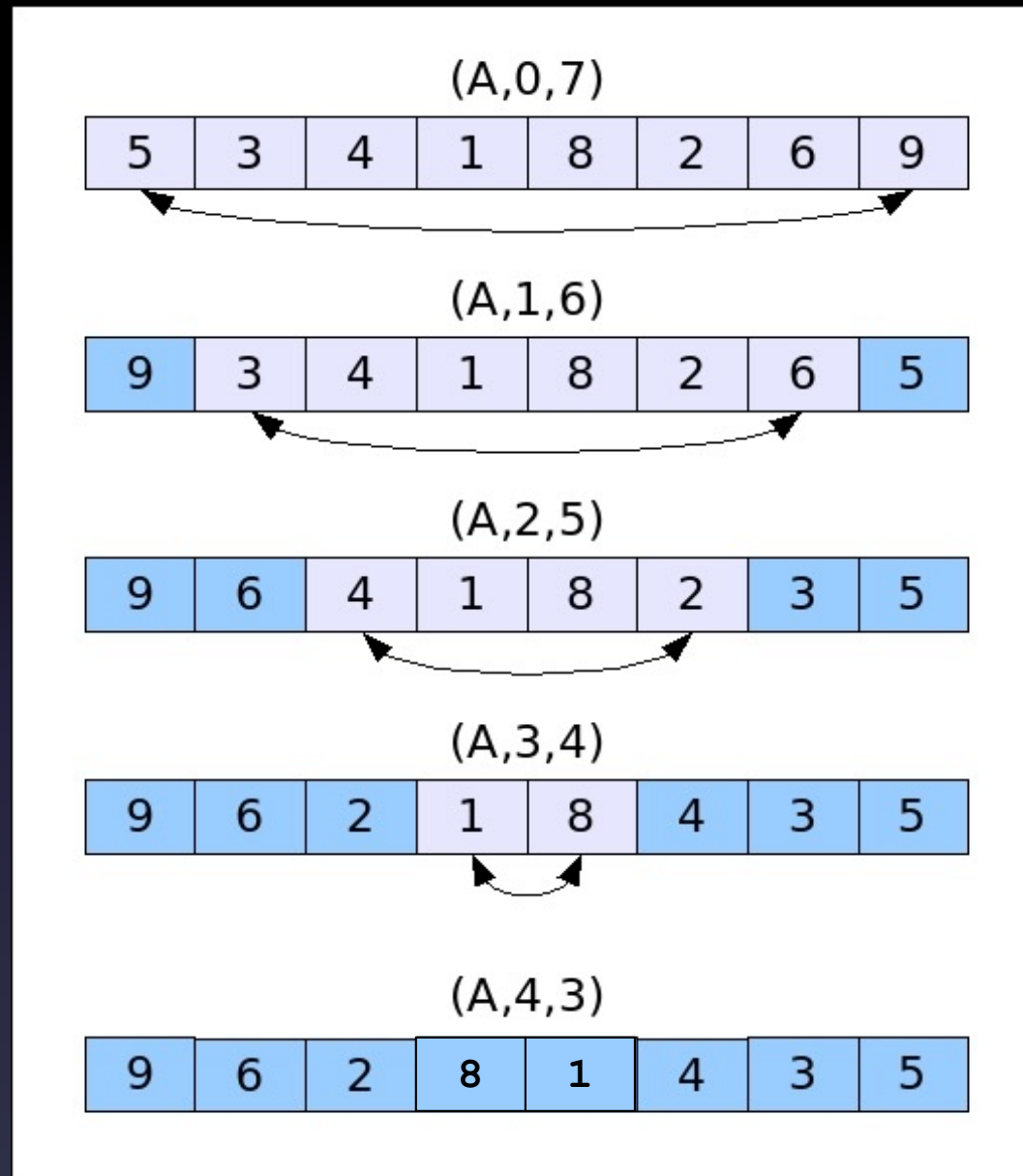
(Can you spot any bug?)

Example: ReverseArray

- ReverseArray(A, i, j)

```
public void ReverseArray(int[] A, int i, int j) {  
    if (i < j) {  
        int tmp = A[i];  
        A[i] = A[j];  
        A[j] = tmp;  
        ReverseArray(A, i+1, j-1);  
    }  
}
```

Example: ReverseArray



Defining Arguments for Recursion

- Recursive methods often need different arguments than non-recursive approaches
- Recursive methods may require **additional parameters**
- We defined array reversal as **ReverseArray(A, i, j)** not **ReverseArray(A)**
Why?

Defining Arguments for Recursion

- Operands are passed forward via parameters
- Simple case
 - result of recursion is passed back via *return*
- In other cases
 - Target object is also passed as parameter
 - Results of recursive calls may
 - affect target object
 - be passed back via *return*

Tail Recursion

- Tail recursion occurs when a linearly recursive method makes its recursive call as its last step
- Easily converted into iterative forms

```
public void ReverseArray( int[] A, int i, int j ) {
```

```
    if ( i < j ) {  
        int tmp = A[ i ];  
        A[ i ] = A[ j ];  
        A[ j ] = tmp;  
        ReverseArray( A, i + 1, j - 1 );  
    }  
}
```

```
public void ReverseArray2( int[] A, int i, int j ) {
```

```
    while ( i < j ) {  
        int tmp = A[ i ];  
        A[ i ] = A[ j ];  
        A[ j ] = tmp;  
        i += 1; j -= 1;  
    }  
}
```


Tail Recursion

- Which function is more efficient?
— (is there even a difference?)

```
public void ReverseArray( int[] A, int i, int j ) {
```

```
    if ( i < j ) {  
        int tmp = A[ i ];  
        A[ i ] = A[ j ];  
        A[ j ] = tmp;  
        ReverseArray( A, i + 1, j - 1 );  
    }  
}
```

```
public void ReverseArray2( int[] A, int i, int j ) {
```

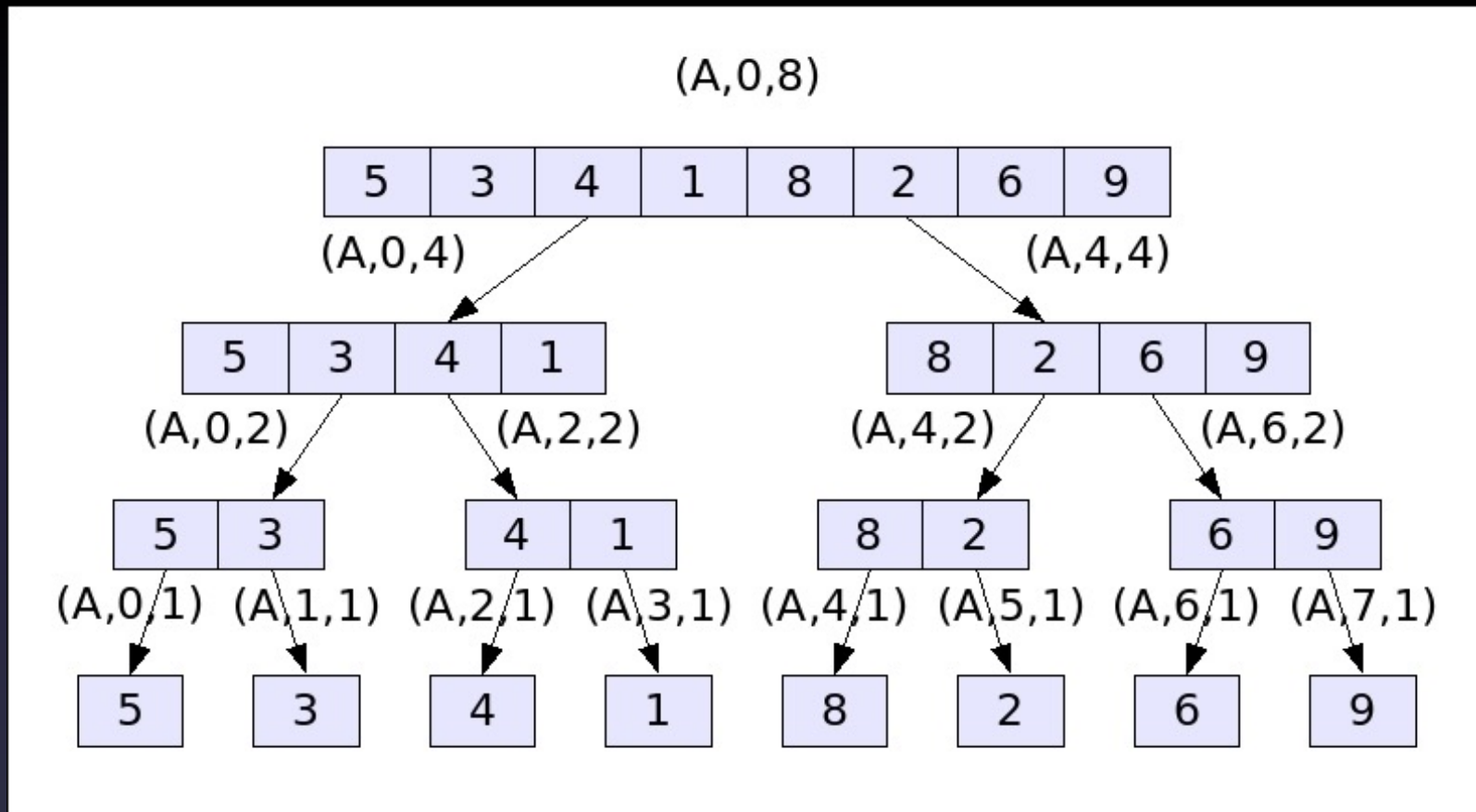
```
    while ( i < j ) {  
        int tmp = A[ i ];  
        A[ i ] = A[ j ];  
        A[ j ] = tmp;  
        i += 1; j -= 1;  
    }  
}
```

Binary Recursion

- Binary recursion occurs whenever there are two calls for each non-base case

```
public int BinarySum(int[] A, int i, int len)
{
    if (len == 1) {
        return A[i];
    } else {
        return BinarySum(A, i, len/2)
            + BinarySum(A, i + len/2, len/2);
    }
}
```

Binary Recursion



Multiple Recursion

- Multiple recursion makes potentially many recursive calls
 - not just one or two
- Multiple recursion is a way of enumerating all possible combinations of a set of elements

Recursion Summary

- Linear Recursion
 - Tail Recursion can easily be mapped to iterative algorithms
- Binary Recursion
 - Recursive method contains two recursive calls (divide and conquer)
- Multiple Recursion

Lecture Summary

- Arrays
 - Fast access, slow to increase size (requires copy)
- Linked lists
 - Fast access to head and tail, slow random access
- Recursion

Feel Like Programming?

- Write a recursive algorithm that will output all the DNA kmers for $k=3$. Subsets of a set of k elements (without repeating subsets).
- The character set is ACGT and kmers of length 3 look like this: AAA, AAC, AAG, AAT, ACA, ACC,..., ATC, ATG, ATT, CAA, ... , TTA, TTC, TTG, TTT
(hint – suppose that you know how make all subsets of $n-1$ elements)
- Implement it in Java!

Feel Like Programming?

- Describe a recursive algorithm that will check if an array A of integers contains an integer $A[i]$ that is the sum of two integers that appear earlier in A , that is, such that $A[i] = A[j] + A[k]$ for $j, k < i$ such that $j \neq k$
- Implement it in Java!

- A recursive algorithm that checks if an array A of integers contains an integer $A[i]$ that is the sum of two integers that appear earlier in A , here is a step-by-step outline:

1. Base Case:

1. If the array has fewer than 3 elements, return false because it's impossible to have an element that is the sum of two distinct earlier elements.

2. Recursive Case:

1. Start with the last element $A[i]$ and check if there are two distinct elements $A[j]$ and $A[k]$ (where $j < k < i$) such that $A[i] = A[j] + A[k]$.
2. If such elements exist, return true.
3. If not, make a recursive call to the function excluding the last element, i.e., check for the subarray $A[0 \dots i-1]$.

```
public boolean containsSumPair(int[] A, int i) {  
    // Base Case: If there are fewer than 3 elements, return false.  
    if (i < 2) {  
        return false;  
    }  
  
    // Check if A[i] is the sum of any two earlier elements.  
    for (int j = 0; j < i; j++) {  
        for (int k = j + 1; k < i; k++) {  
            if (A[i] == A[j] + A[k]) {  
                return true;  
            }  
        }  
    }  
}  
  
    // Recur for the array excluding the last element.  
    return containsSumPair(A, i - 1);  
}  
  
// This function initiates the recursion.  
public boolean containsSumPair(int[] A) {  
    return containsSumPair(A, A.length - 1);  
}
```